

UNIDAD 2

Creación de bots

Diferencia entre bots e IA

muy común confundirlos porque ambos automatizan tareas, pero la diferencia principal está en la **capacidad de aprender y adaptarse**.

Para ponerlo de forma sencilla: un bot sigue **reglas**, mientras que la IA sigue **patrones**.

Aquí tienes las diferencias clave explicadas al detalle:

El Bot Tradicional (Basado en reglas)

Un bot es un software diseñado para realizar tareas repetitivas y automatizadas siguiendo un guion o un conjunto de instrucciones estrictas escritas por un programador.

- **Cómo funciona:** Si pasa **A**, entonces hace **B**. Si el usuario se sale del guion, el bot se pierde y no sabe qué responder.
- **Capacidad de aprendizaje:** Cero. No aprende de sus errores ni de las interacciones pasadas. Si tiene un fallo, un humano debe entrar al código y corregirlo.
- **Ejemplo común:** Los típicos chats de atención al cliente de los bancos que te dan un menú con opciones numéricas ("Presione 1 para saldo, 2 para transferir"). Si le escribes un poema o le cuentas un problema complejo, te responderá con un mensaje de error.

La Inteligencia Artificial (Basada en aprendizaje)

La IA es un sistema diseñado para simular procesos de inteligencia humana, como el razonamiento, el aprendizaje y la resolución de problemas complejos.

- **Cómo funciona:** No necesita un guion rígido. Analiza grandes cantidades de datos, entiende el contexto, interpreta la intención del usuario y genera una respuesta propia.
- **Capacidad de aprendizaje:** Alta. A través del *Machine Learning*, el sistema mejora su precisión a medida que interactúa con más datos y usuarios.
- **Ejemplo común:** Un asistente que no solo entiende lo que dices, sino cómo lo dices, puede detectar si estás enojado, redactar un correo desde cero con el tono que le pidas o programar una aplicación entera.

FLUJOS DE CONVERSACION EN UN CHATBOT

Para diseñar flujos de conversación (*conversational flows*) efectivos para un chatbot, el secreto está en **equilibrar la claridad de un bot tradicional con la flexibilidad de la inteligencia artificial**. Un buen flujo guía al usuario hacia su objetivo sin hacerlo sentir atrapado en un laberinto de opciones.

Aquí tienes una guía paso a paso, las mejores estructuras y las reglas de oro para diseñar flujos que realmente funcionen.

1. Tipos de Estructuras para Flujos de Conversación

Dependiendo de la complejidad de tu chatbot, puedes estructurar el flujo de tres maneras:

A. Flujo Lineal o de Árbol de Decisión (Basado en Reglas)

Es el más sencillo. Sigue una estructura rígida de tipo "pregunta-respuesta" mediante botones u opciones fijas. Es ideal para procesos estructurados (como agendar una cita o consultar un saldo).

```
[Saludo]
↓
[Menú de Opciones: 1. Comprar / 2. Soporte]
↓ (Si elige 2)
[¿Cuál es tu problema? A. Envío / B. Factura]
B. Flujo Basado en Intenciones (NLU / IA)
```

Aquí el usuario escribe libremente. El motor de Inteligencia Artificial (Procesamiento de Lenguaje Natural) detecta la **intención** de la frase, extrae las **entidades** (datos clave) y salta directamente al nodo del flujo correspondiente.

- *Usuario:* "Quiero cancelar el pedido que hice ayer."
- *IA detecta:* Intención = `cancelar_pedido` | Entidad = `fecha: ayer`.
- *Acción:* Salta directo al flujo de cancelación sin pasar por menús previos.

C. Flujo Híbrido (El recomendado)

Combina lo mejor de ambos mundos. Deja que la IA entienda la primera frase abierta del usuario para identificar qué quiere hacer, pero una vez dentro del proceso, utiliza un flujo guiado (con botones o preguntas específicas) para recolectar los datos de forma rápida y sin errores.

2. Anatomía de un Flujo de Conversación Completo

Un flujo estándar debe cubrir las siguientes etapas desde que el usuario abre el chat hasta que se despide:

1. El Saludo e Inicio (Onboarding)

Define el tono del bot, establece expectativas claras (qué puede y qué no puede hacer) y da la iniciativa para empezar.

- *Ejemplo:* "¡Hola! Soy [Nombre del Bot], tu asistente virtual. Te puedo ayudar a rastrear tu pedido, gestionar devoluciones o agendar una asesoría. ¿Qué te gustaría hacer hoy?" (Acompañado de botones con esas opciones).

2. Recolección de Datos (Slots Filling)

Si el proceso requiere información (un correo, un ID de estudiante, un número de orden), el bot debe pedir un dato a la vez.

- *Tip de diseño:* Si el usuario da un dato erróneo (un correo sin el "@"), el flujo debe incluir un **bucle de validación** que explique amablemente el error y vuelva a pedir el dato.

3. Confirmación y Acción

Antes de ejecutar una acción importante (como hacer un cobro o cancelar un registro), el bot debe resumir la información y pedir confirmación.

- *Ejemplo:* "Perfecto. Voy a agendar tu cita para el lunes a las 15:00. ¿Es correcto?" [Sí, confirmar] [No, cambiar].

4. Cierre y Salida

Agradece al usuario, confirma que el objetivo se cumplió y cierra la sesión de forma limpia.

- *Ejemplo:* "Tu solicitud ha sido procesada. ¿Hay algo más en lo que te pueda ayudar hoy?"

3. El Flujo de Respaldo (Fallback Flow): El salvavidas del bot

Este es el nodo más importante del diseño. ¿Qué pasa si el usuario dice algo que el bot no entiende o se sale por completo del guion? Entra el **Fallback**.

- **Primer error:** "Lo siento, no logré entender eso. ¿Podrías decírmelo de otra forma o elegir una de estas opciones?"
- **Segundo error consecutivo:** No repitas el mismo mensaje. Ofrece una alternativa directa o el desvío a un humano: "Sigo sin entenderlo del todo. Para no hacerte perder tiempo, ¿prefieres que te comunique con un agente humano?"

4. Reglas de Oro para Diseñar Flujos Exitosos

- **Une las instrucciones complejas en bloques compactos:** Evita que el bot envíe 5 mensajes seguidos de una sola línea haciendo vibrar el teléfono del usuario. Es mejor agrupar la información en un párrafo bien estructurado y directo.
- **Mensajes cortos y escaneables:** Usa negritas para resaltar palabras clave (fechas, nombres, acciones) y viñetas si tienes que listar requisitos o pasos.
- **Siempre da una salida:** Nunca dejes un flujo en un "callejón sin salida". Cada respuesta del bot debe terminar con una pregunta, opciones o un botón para volver al menú principal.
- **Mantén la personalidad consistente:** Si tu bot es un asistente académico o corporativo, debe mantener un tono profesional, claro y empático en cada paso del flujo; si es juvenil, puede usar un tono más fresco.

Chatbot clásico con scratch

1. La lógica detrás del Bot en Scratch

Para que el chatbot parezca "inteligente", usaremos una estructura condicional. El robot hará una pregunta, guardará la respuesta del usuario en una variable integrada llamada **respuesta** y luego evaluará:

- **SI** la respuesta contiene la palabra clave X → Dice la respuesta X.
- **SI NO** → Evalúa la siguiente palabra clave o activa un mensaje de error (*fallback*).

2. Los Bloques Clave que vas a usar

En la paleta de bloques de Scratch, necesitarás ubicar principalmente estos cuatro:

1. **preguntar [¿...?] y esperar** (Categoría **Sensores**): Abre la caja de texto para que el usuario escriba.
2. **respuesta** (Categoría **Sensores**): Bloque ovalado que almacena lo último que el usuario escribió.
3. **¿ [texto] contiene [texto] ?** (Categoría **Operadores**): Este bloque es el "cerebro"; verifica si una palabra está dentro de la respuesta.
4. **si <...> entonces / si no** (Categoría **Control**): La estructura que decide qué camino toma la conversación.

3. Estructura del Código (Paso a Paso)

Imagina que estás programando un asistente para un juego o un personaje robot. El código principal dentro del objeto (el *Sprite*) debería verse estructurado de la siguiente manera:

Paso 1: El Saludo e Inicio (Onboarding)

Al presionar la bandera verde, el bot se presenta y le dice al usuario qué opciones o comandos puede entender para guiar la conversación.

Fragmento de código

```
al hacer clic en bandera verde
decir [¡Hola! Soy Bot-3000, tu asistente virtual.] por (3) segundos
preguntar [¿De qué te gustaría hablar hoy? (Escribe: "juegos", "clima" o "saludo")] y esperar
```

Paso 2: El Flujo de Decisiones (Árbol de Condiciones)

Aquí es donde conectamos las opciones. Usaremos bloques si / si no anidados (uno dentro de otro) para ir descartando las respuestas del usuario.

Fragmento de código

```
si < (respuesta) contiene (saludo) ? > entonces
    decir [¡Hola humano! Espero que estés teniendo un excelente día.] por (3) segundos
si no
    si < (respuesta) contiene (juegos) ? > entonces
        preguntar [A mí me encantan los videojuegos de plataformas. ¿Cuál es tu juego favorito?] y esperar
        decir [¡Buenísimo! He oído que ese juego es genial.] por (3) segundos
    si no
        si < (respuesta) contiene (clima) ? > entonces
            decir [No tengo ventanas, pero mis sensores dicen que el día está perfecto para programar.] por (3) segundos
        si no
            # Este es el bloque de Fallback (si no entiende nada de lo anterior)
            decir [Lo siento, mi sistema no reconoce esa palabra. Intenta con "juegos" o "clima".] por (3) segundos
        fin
    fin
fin
```